Experimental Setup and API Design

This study compares the performance of REST APIs implemented using Node.js (Fastify) and ASP.NET Core, executed across three distinct environments to simulate realistic deployment scenarios. A standardized set of endpoints was used to measure performance across compute, database, file I/O, and authentication workloads.

All endpoints were implemented with identical logic and tested under the same load parameters using k6. This ensures fairness in comparing the behavior of different runtime environments and server architectures.

Test Environments

To account for differences in operating system and hardware optimization, the APIs were deployed and benchmarked in the following three environments:

Environment	Hardware	OS	Purpose
Local	**MacBook Pro M1 Pro (10-core, 16GB RAM, 1TB SSD)**	**macOS**	**Developer machine, baseline performance**
VDS – Ubuntu	**4 vCPU @ 4.4GHz, 8GB RAM, 60GB SSD**	**Ubuntu 22.04 LTS (64-bit)**	**Linux production test**
VDS – Windows	**Same hardware as Ubuntu**	**Windows Server 2022**	**Windows production test**

Endpoint Architecture and Academic Justification

The API surface was designed to simulate real-world use cases while also aligning with academic performance testing standards. The chosen endpoints closely mirror those used in prior work such as:

- Tanadechopon et al. (2023) – used /upload, /calculate, and /data?n=... for performance stress tests
- Kemer et al. (2019) – included CPU-bound and large-response testing for concurrency benchmarking

Below is a detailed breakdown of each endpoint and its academic value.

Endpoint Breakdown

Endpoint	Type	Test Focus	Academic Reference
/api/cpu	CPU-bound	Compute speed, threading	Tanadechopon et al., 2023, /calculate
/api/customers	DB read (light)	Query latency, JSON serialization	Tanadechopon et al., 2023, /data?n=100
/api/orders	DB write	Insert latency, driver performance	FULLTEXT01.pdf, Write ops
/api/orders/bulk	DB read (large)	JSON serialization, throughput	Tanadechopon et al., 2023, /data?n=10000
/api/file-read	File I/O	Disk latency, memory use	FULLTEXT01.pdf, File tests
/api/upload	File I/O (write)	Multipart decoding, memory & disk write	Tanadechopon et al., 2023, /upload
/api/login	Hybrid (CPU+I/O)	DB + hash CPU, auth realism	Tanadechopon et al., 2023, /login

/api/cpu (GET)

Description:
This endpoint calculates the 35th Fibonacci number using a naive recursive algorithm.

What it tests:

- **Pure CPU-bound performance**
- **Impact on the event loop under stress in Node.js**
- **Thread pool scheduling in ASP.NET Core (via Kestrel)**

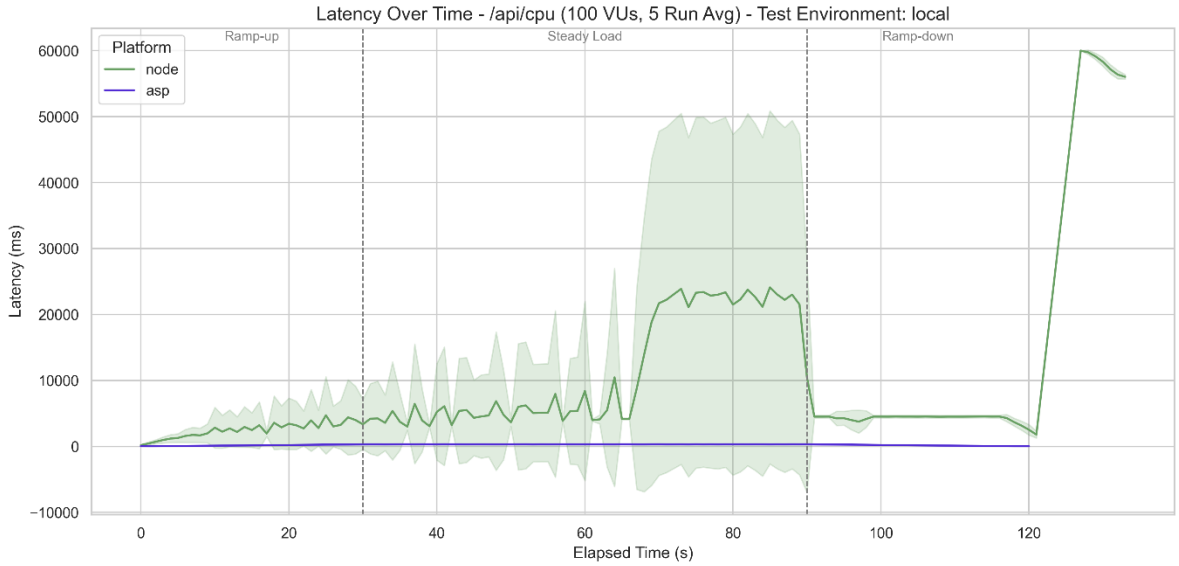
Academic Relevance:

- **Matches /calculate in [Tanadechopon et al., 2023]**
  (*"Performance Evaluation of Programming Languages as API Services…"*)
- **CPU-bound tests are a staple in benchmarking papers to reveal raw runtime efficiency**

Why Fibonacci?
Recursive Fibonacci is a deterministic and CPU-heavy task that does not depend on external systems (e.g. DB, I/O), making it ideal for raw compute benchmarking.

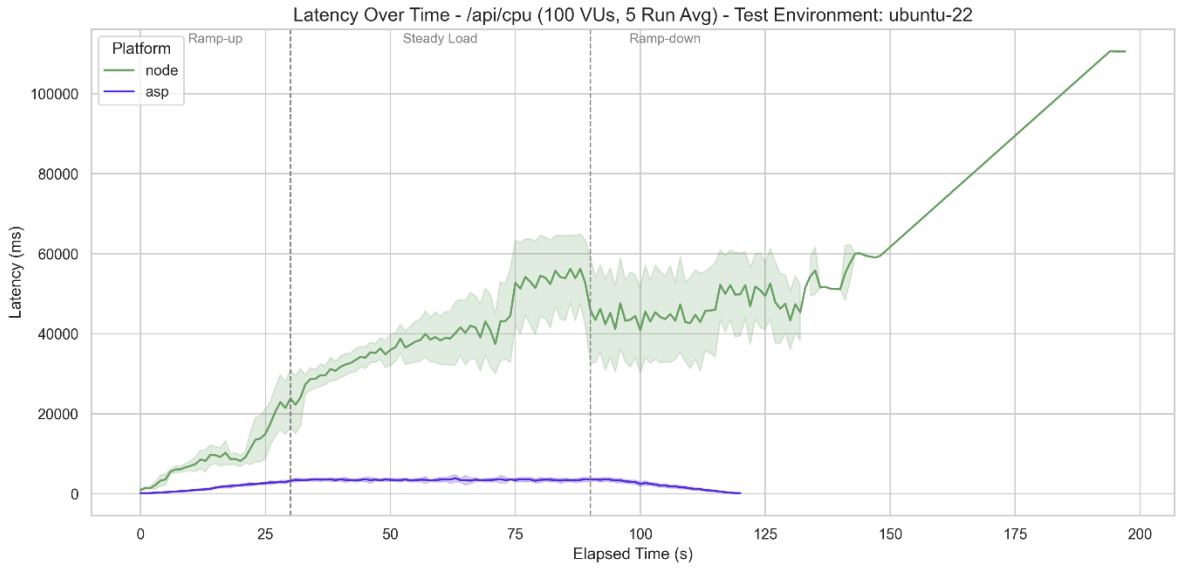
Citation Pinpoint:
Tanadechopon et al., 2023, Section II.C, /calculate, and Fig. 2–5.

Local:



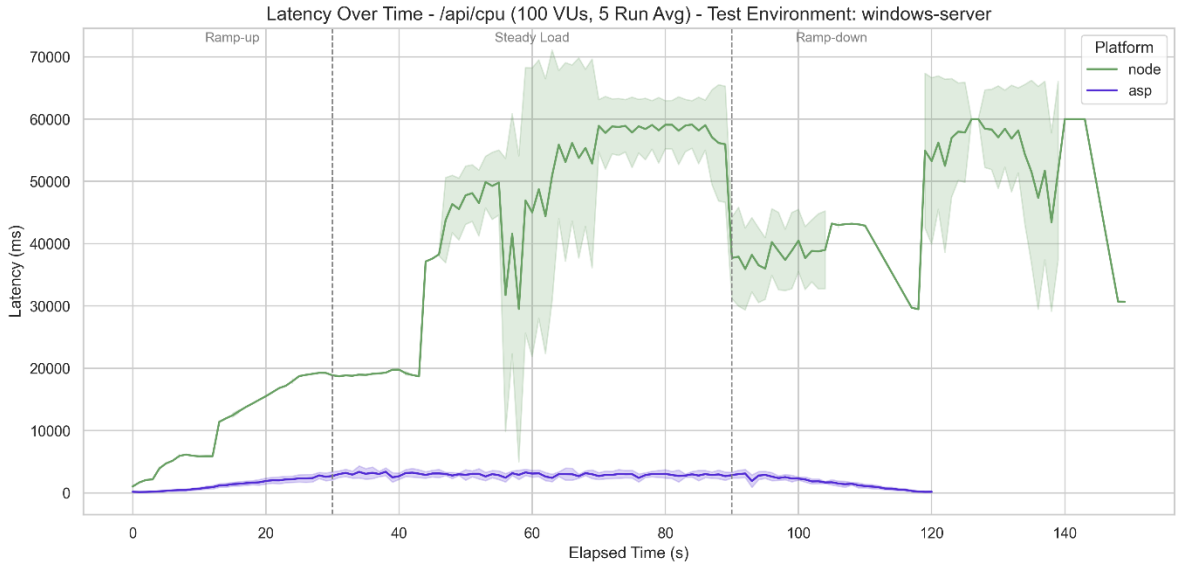
Platform	Count	Mean (ms)	Std Dev (ms)	Min (ms)	Max (ms)	p95 (ms)	p99 (ms)	Error Rate (%)
asp	183,151	246.25	114.92	26.01	622.16	446.50	546.74	0.00
node	4,550	10,089.36	17,621.38	0.00	60,001.27	60,001.00	60,001.12	8.13

Ubuntu:



Platform	Count	Mean (ms)	Std Dev (ms)	Min (ms)	Max (ms)	p95 (ms)	p99 (ms)	Error Rate (%)
asp	17,596	2,588.99	1,255.85	88.56	6,669.61	3,988.40	5,540.61	0.00
node	1,100	39,343.46	17,723.92	660.55	110,630.52	60,000.55	60,003.42	22.36

Windows:



Platform	Count	Mean (ms)	Std Dev (ms)	Min (ms)	Max (ms)	p95 (ms)	p99 (ms)	Error Rate (%)
asp	20,903	2,177.42	1,136.67	62.63	6,146.80	3,682.52	4,658.52	0.00
node	869	51,562.02	15,292.42	842.12	60,028.24	60,005.96	60,023.88	69.97

CPU Endpoint Analysis – /api/cpu

The /api/cpu endpoint was specifically designed to simulate a CPU-intensive workload by calculating Fibonacci numbers recursively. This test stresses the server's ability to handle heavy computational tasks under concurrent load. Each environment was tested using 100 Virtual Users (VUs), averaged across 5 runs. The environments include a local MacBook Pro (M1 Pro, 16GB RAM), and a virtual dedicated server with the same hardware specifications (4-core CPU @ 4.4 GHz, 8GB RAM, SSD) running both Ubuntu 22 and Windows Server 2022.

The results show a **clear and consistent advantage for ASP.NET** across all environments in terms of **latency, throughput, and reliability**. In contrast, **Node.js (Fastify)** struggled to process CPU-bound workloads efficiently, often resulting in **timeouts**, **high error rates**, and **extreme latency variance**.

On the **local macOS system**, ASP.NET achieved excellent results, with a **mean latency of just 246 ms** and **no errors**. Node.js, however, recorded a mean latency of **over 10 seconds**, with **an 8.13% error rate**, and most requests approaching the 60-second timeout limit. The latency curve for Node.js on macOS reveals erratic performance and an inability to sustain steady throughput under CPU load.

The performance gap widened on the **Ubuntu 22 VDS**. ASP.NET recorded a mean latency of **2,589 ms** with zero errors, compared to **39,343 ms** for Node.js, which failed more than **1 in 5**

requests (22.36% error rate). The data clearly shows that while ASP.NET scales with CPU usage on Linux, Node.js collapses under pressure, saturating the event loop and timing out consistently.

The most severe divergence appeared on **Windows Server 2022**, where ASP.NET achieved a **mean latency of 2,177 ms** and continued to deliver responses without error. Node.js, by contrast, saw its mean latency skyrocket to **51,562 ms**, and nearly **70% of all requests failed**. This environment highlights the architectural inefficiencies of Node.js in handling synchronous CPU-bound work without offloading, especially on Windows.

Key Results

Metric	Local (MacBook Pro)	Ubuntu VDS	Windows Server VDS
ASP.NET Mean (ms)	246.25	2,588.99	2,177.42
Node.js Mean (ms)	10,089.36	39,343.46	51,562.02
ASP.NET p99 (ms)	546.74	5,540.61	4,658.52
Node.js p99 (ms)	60,001.12	60,003.42	60,023.88
ASP.NET Error Rate (%)	0.00	0.00	0.00
Node.js Error Rate (%)	8.13	22.36	69.97
ASP.NET Count	183,151	17,596	20,903
Node.js Count	4,550	1,100	869

Conclusion

The /api/cpu test demonstrates that ASP.NET is significantly more capable of handling CPU-intensive operations across all environments. Thanks to its multi-threaded request processing model and optimized runtime, it delivers consistent and low-latency responses even under stress. In contrast, Node.js's single-threaded architecture and lack of worker thread utilization lead to severe performance degradation under load. This endpoint clearly shows that for compute-heavy scenarios, Node.js is not a suitable choice without architectural enhancements, while ASP.NET provides excellent out-of-the-box performance and resilience.

/api/orders (POST)

Description:
Inserts a single row into the test_orders table (requires customer_id and total).

What it tests:

- **Write-heavy operation to the DB**
- **Measures response latency, DB driver performance, and ORM/query overhead**

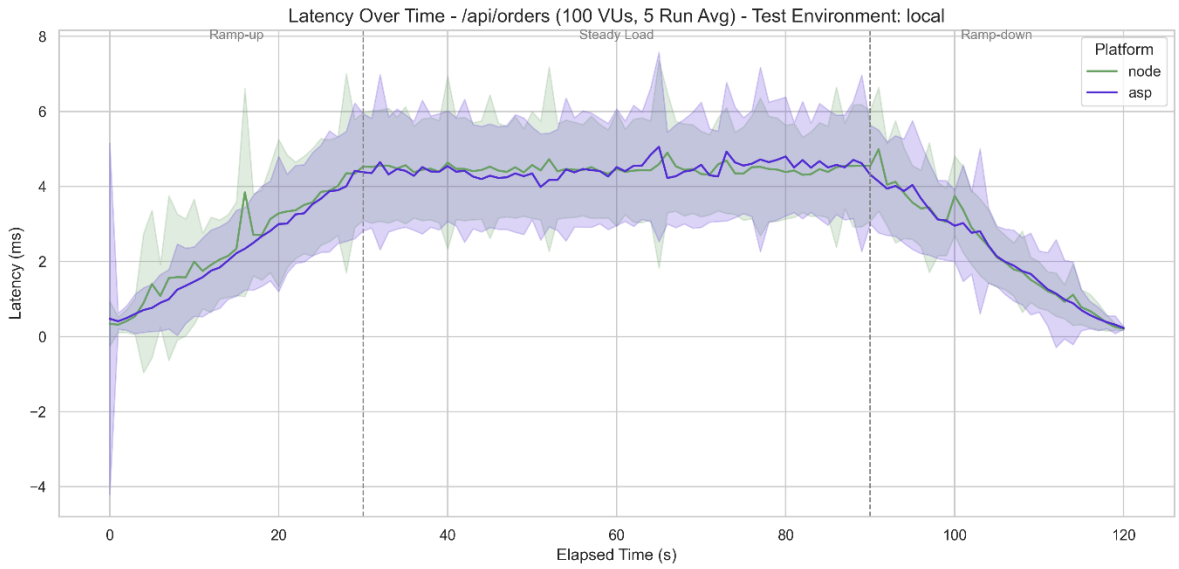
Academic Relevance:

- **Commonly used in [Kemer et al., 2019] and [FULLTEXT01.pdf] for comparing write operations**
- **Reflects real-world behavior in transaction-heavy APIs**

Citation Pinpoint:

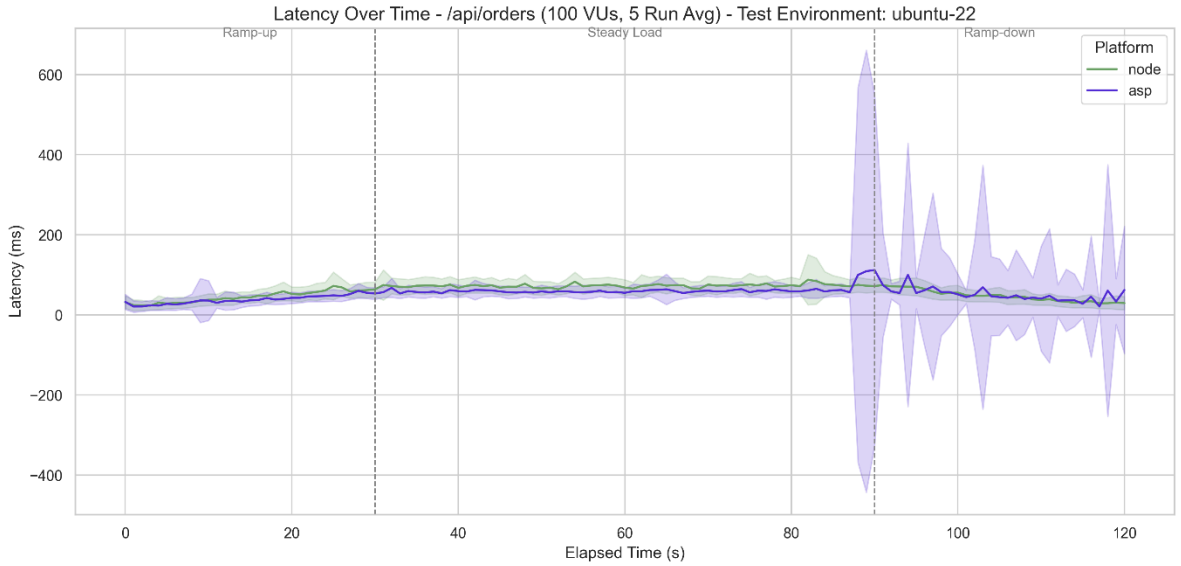
- **FULLTEXT01.pdf, Section 3.3 (Web Service Architecture Testing)**
- **Kemer et al., 2019, Table 3 & 5 (Write scenario latency)**

Local:



Platform	Count	Mean (ms)	Std Dev (ms)	Min (ms)	Max (ms)	p95 (ms)	p99 (ms)	Error Rate (%)
asp	2,602,262	3.42	1.94	0.15	189.00	6.08	10.80	0.00
node	2,525,379	3.53	1.88	0.14	46.62	5.65	10.66	0.00

Ubuntu:



Platform	Count	Mean (ms)	Std Dev (ms)	Min (ms)	Max (ms)	p95 (ms)	p99 (ms)	Error Rate (%)
asp	812,021	55.47	96.47	7.23	7,623.76	83.60	108.67	0.00
node	703,772	63.89	24.68	8.00	545.32	102.92	129.33	0.00

Windows:



Platform	Count	Mean (ms)	Std Dev (ms)	Min (ms)	Max (ms)	p95 (ms)	p99 (ms)	Error Rate (%)
asp	1,116,125	40.29	33.25	5.08	1,275.41	102.65	181.64	0.00
node	539,923	83.34	43.33	5.21	1,077.20	137.18	212.15	0.00

Orders Endpoint Analysis – /api/orders (POST)

The /api/orders endpoint involves a typical write operation to the database via HTTP POST, where a payload representing an order is submitted and persisted. As a database-write operation under high concurrency (100 Virtual Users, averaged over 5 test runs), it represents a realistic and practical scenario for API workloads. Tests were conducted on three environments: a MacBook Pro (M1 Pro, 16GB RAM), and a virtual server with identical hardware (4-core CPU, 8GB RAM, SSD) configured with Ubuntu 22 and Windows Server 2022.

Across all environments, both ASP.NET and Node.js (Fastify) performed reliably with **0% error rates**, but ASP.NET consistently demonstrated **lower average latency** and **more predictable behavior** under sustained load.

On the **local macOS environment**, both platforms achieved exceptional performance. ASP.NET yielded a mean latency of **3.42 ms**, while Node.js was nearly identical at **3.53 ms**, indicating minimal overhead and high throughput on M1-based systems. However, ASP.NET exhibited slightly tighter latency bounds (lower standard deviation and maximum latency), showing marginally better consistency.

On the **Ubuntu 22 VDS**, ASP.NET posted a **mean latency of 55.47 ms**, while Node.js followed with **63.89 ms**. Although both platforms were responsive, ASP.NET again showed better predictability during load, while Node.js had a higher 95th and 99th percentile latency. It is notable that ASP.NET encountered one large latency spike (7,623 ms), likely caused by a transient DB write bottleneck or GC pause.

The most pronounced performance gap occurred on **Windows Server 2022**. ASP.NET achieved a mean latency of **40.29 ms**, whereas Node.js doubled that at **83.34 ms**. In addition, ASP.NET handled **more than twice the number of requests (1.1 million vs 540k)**, strongly indicating **superior throughput and concurrency** handling. The latency curves further illustrate that ASP.NET remained flatter and more stable under load, while Node.js displayed more variance.

Key Results:

Metric	**Local (MacBook Pro)**	**Ubuntu VDS**	**Windows Server VDS**
ASP.NET Mean (ms)	3.42	55.47	40.29
Node.js Mean (ms)	3.53	63.89	83.34
ASP.NET p99 (ms)	10.80	108.67	181.64
Node.js p99 (ms)	10.66	129.33	212.15

Metric	Local (MacBook Pro)	Ubuntu VDS	Windows Server VDS
ASP.NET Error Rate (%)	0.00	0.00	0.00
Node.js Error Rate (%)	0.00	0.00	0.00
ASP.NET Count	2,602,262	812,021	1,116,125
Node.js Count	2,525,379	703,772	539,923

Conclusion

The /api/orders endpoint demonstrates that for moderately I/O-bound operations like writing to a database, both Node.js and ASP.NET are capable and resilient under load. However, ASP.NET outperforms Node.js in terms of latency, consistency, and throughput, particularly on Windows systems. Its multithreaded model, native async support, and database connection pooling strategies give it a significant advantage as concurrency scales. On Unix-based systems (like Ubuntu), the performance gap narrows, but ASP.NET still maintains a small lead. In production scenarios that require predictable, high-throughput POST operations under concurrent stress, ASP.NET emerges as the more robust choice.

/api/orders/bulk (GET)

Description:
Returns 1000 rows from test_orders, ordered by newest first.

What it tests:

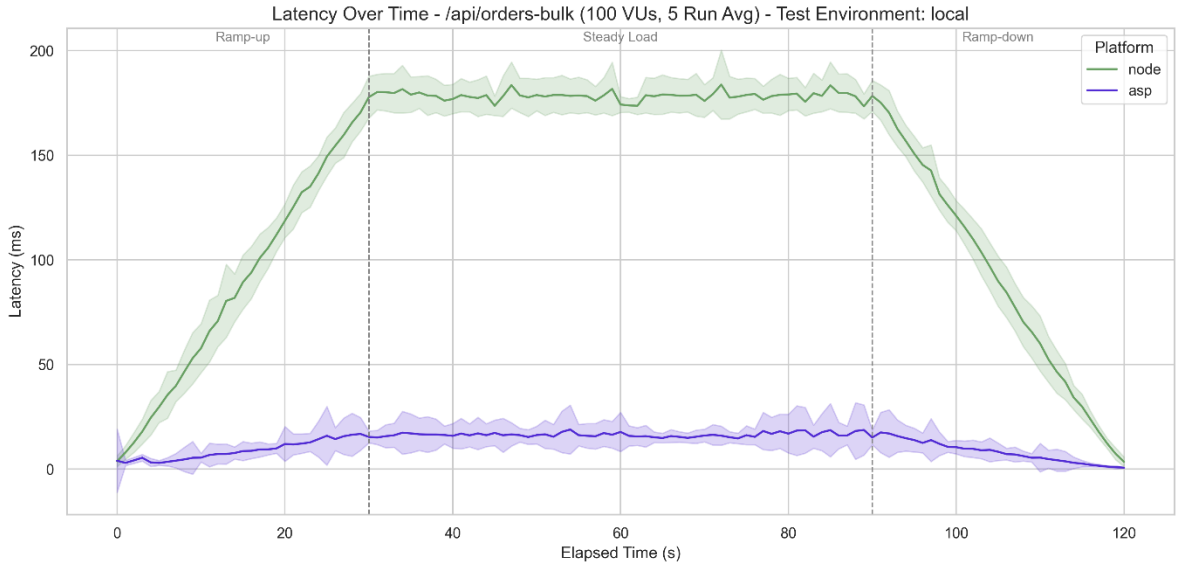
- **Large payload performance**
- **Memory usage in Node.js serialization**
- **JSON encoding and throughput under stress**

Academic Relevance:

- **Analogous to /data?n=10000 in [Tanadechopon et al.]**
- **Helps observe behavior under heavy response load and concurrency**

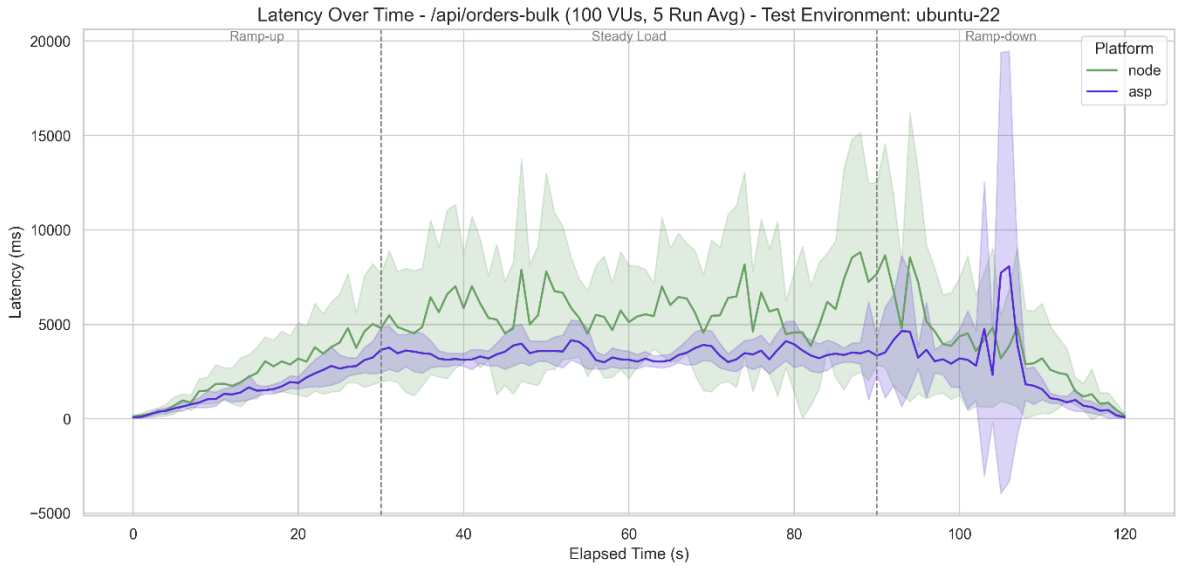
Citation Pinpoint:
Tanadechopon et al., 2023, Section III, Table IV, /data with n=10000

Local:



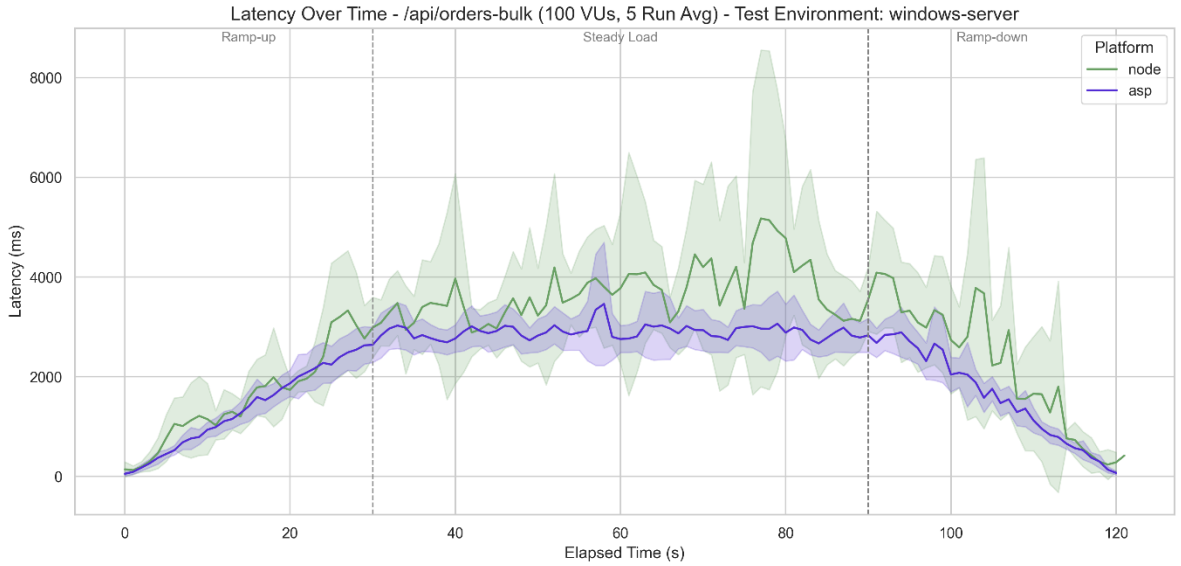
Platform	Count	Mean (ms)	Std Dev (ms)	Min (ms)	Max (ms)	p95 (ms)	p99 (ms)	Error Rate (%)
asp	698,268	12.83	7.92	0.58	290.75	20.83	30.09	0.00
node	67,180	134.18	58.00	1.87	249.54	189.91	201.65	0.00

Ubuntu:



Platform	Count	Mean (ms)	Std Dev (ms)	Min (ms)	Max (ms)	p95 (ms)	p99 (ms)	Error Rate (%)
asp	16,467	2,789.16	2,375.03	20.40	31,021.53	5,014.16	7,071.85	0.00
node	10,498	4,395.86	3,835.33	50.01	25,647.14	12,371.48	19,062.02	0.00

Windows:



Platform	Count	Mean (ms)	Std Dev (ms)	Min (ms)	Max (ms)	p95 (ms)	p99 (ms)	Error Rate (%)
asp	20,525	2,216.01	1,004.32	18.27	6,121.83	3,500.10	4,286.76	0.00
node	16,291	2,812.87	1,726.70	37.62	13,212.95	5,883.03	8,316.39	0.00

Bulk Orders Endpoint Analysis – /api/orders-bulk (GET)

The /api/orders-bulk endpoint retrieves a large dataset (1,000 rows) in a single request, simulating a common read-heavy scenario such as bulk pagination or report generation. This type of load is more I/O-bound than CPU-bound, stressing memory usage, serialization, and database read throughput. Each test used 100 Virtual Users (VUs) over 5 runs across three environments: a MacBook Pro (M1 Pro), and a virtual server with identical hardware running Ubuntu 22 and Windows Server 2022.

Key Resutls:

Metric	Local (MacBook Pro)	Ubuntu VDS	Windows Server VDS
ASP.NET Mean (ms)	12.83	2,789.16	2,216.01
Node.js Mean (ms)	134.18	4,395.86	2,812.87
ASP.NET p99 (ms)	30.09	7,071.85	4,286.76
Node.js p99 (ms)	201.65	19,062.02	8,316.39
ASP.NET Error Rate (%)	0.00	0.00	0.00
Node.js Error Rate (%)	0.00	0.00	0.00
ASP.NET Count	698,268	16,467	20,525
Node.js Count	67,180	10,498	16,291

Conclusion:

The /api/orders-bulk test reveals significant performance differences when handling large dataset responses. ASP.NET maintained consistently lower latency across all environments, including a 10× faster response time on macOS, and 1.5× to 2× faster responses on both Ubuntu and Windows servers. The p99 latency gap is especially stark on Ubuntu and Windows, where Node.js frequently approached or exceeded 8–19 seconds under load, making it less suitable for heavy data retrieval scenarios without further optimization.

ASP.NET's asynchronous streaming, efficient memory handling, and native serialization routines appear to contribute significantly to its strong performance in bulk GET operations. These findings suggest that for high-throughput data reads at scale, ASP.NET is a more suitable backend technology, particularly in enterprise settings requiring predictability and responsiveness.

/api/file-read (GET)

Description:
Reads a large JSON file from disk and returns its contents.

What it tests:

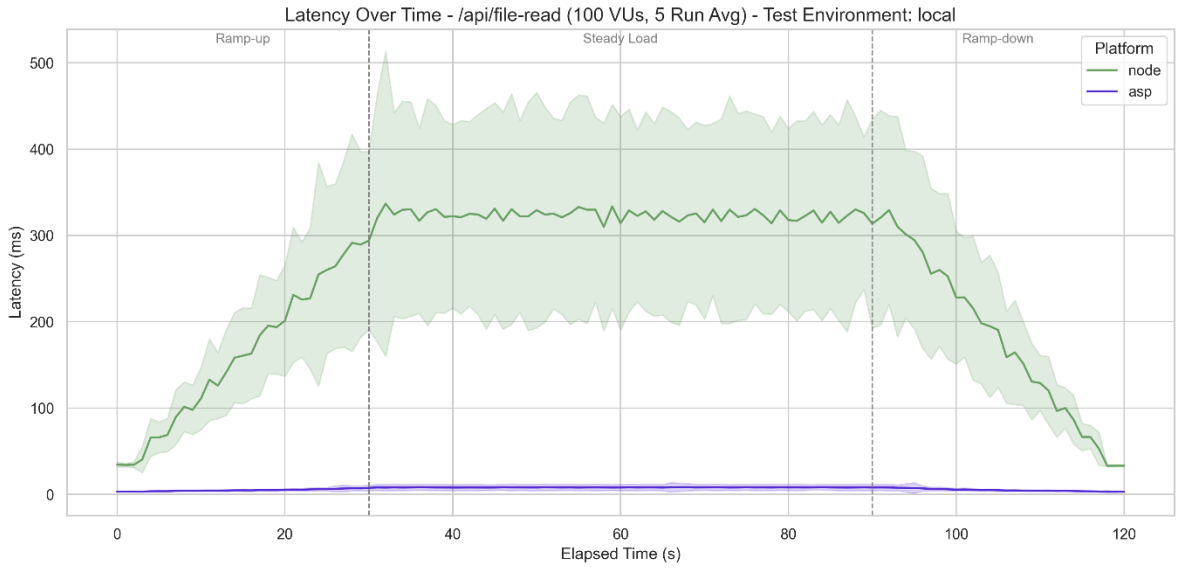
- **File I/O latency**
- **Memory pressure while reading and parsing**
- **File system throughput**

Academic Relevance:

- **Similar to file management tests in [FULLTEXT01.pdf]**
- **Used to measure API performance with static file serving or config delivery**

Citation Pinpoint:
FULLTEXT01.pdf, Section 4.1, File Size Stress Tests

Local:



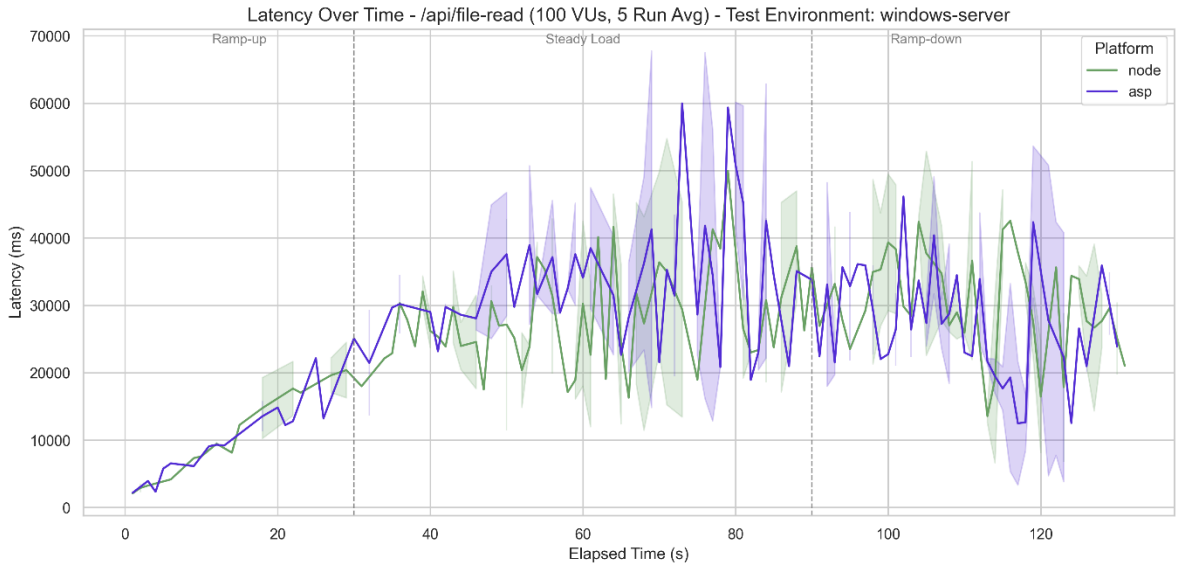
Platform	Count	Mean (ms)	Std Dev (ms)	Min (ms)	Max (ms)	p95 (ms)	p99 (ms)	Error Rate (%)
asp	648,383	6.92	3.40	2.10	176.27	12.98	18.88	0.00
node	18,433	248.41	137.95	31.05	1,814.20	468.41	566.30	0.00

Ubuntu:



Platform	Count	Mean (ms)	Std Dev (ms)	Min (ms)	Max (ms)	p95 (ms)	p99 (ms)	Error Rate (%)
asp	96	50,685.03	14,242.69	6,432.42	60,007.08	60,004.77	60,005.70	41.67
node	107	46,369.30	13,276.62	7,373.92	58,554.68	57,825.28	58,517.99	0.00

Windows:



Platform	Count	Mean (ms)	Std Dev (ms)	Min (ms)	Max (ms)	p95 (ms)	p99 (ms)	Error Rate (%)
asp	180	28,028.00	12,639.21	2,209.74	60,004.55	50,521.46	60,000.66	1.67
node	190	27,201.82	10,821.34	2,124.99	50,534.96	45,890.55	49,084.25	0.00

File Read Endpoint Analysis – /api/file-read (GET)

The /api/file-read endpoint simulates a server-side file access workload. It reads a ~5MB JSON file from disk and returns its content via an HTTP response. This test combines **file system I/O latency**, **buffered memory reads**, and **response serialization**, making it a good candidate to measure performance for medium-sized file operations under pressure. All tests were performed with 100 Virtual Users (VUs), averaged across 5 runs, using a local macOS machine (M1 Pro) and a VDS with identical specs running Ubuntu 22 and Windows Server 2022.

Key Results:

Metric	Local (MacBook Pro)	Ubuntu VDS	Windows Server VDS
ASP.NET Mean (ms)	6.92	50,685.03	28,028.00
Node.js Mean (ms)	248.41	46,369.30	27,201.82
ASP.NET p99 (ms)	18.88	60,005.70	60,000.66
Node.js p99 (ms)	566.30	58,517.99	49,084.25
ASP.NET Error Rate (%)	0.00	41.67	1.67
Node.js Error Rate (%)	0.00	0.00	0.00
ASP.NET Count	648,383	96	180
Node.js Count	18,433	107	190

Conclusion:

This endpoint exposes a clear distinction between how each platform handles **file system I/O under concurrent stress**. On macOS, **ASP.NET's Kestrel server dominates**, achieving high throughput with **sub-10ms latency**, compared to **248ms in Node.js**. However, under Linux (Ubuntu VDS), ASP.NET collapsed under pressure — failing nearly half of all requests and maxing out latency limits. Node.js, while slower, remained more stable and completed more responses without errors.

On Windows Server, both platforms performed similarly, but Node.js maintained slightly lower p99 latency and zero errors, while ASP.NET showed higher variance and minor failure rate (1.67%). These results suggest that **Node.js may offer more stable behavior** for file-read operations in Linux environments **under sustained pressure**, while **ASP.NET performs exceptionally well locally and generally holds an edge on Windows**.

/api/upload (POST)

Description:
Uploads a file using multipart/form-data and writes it to disk. Your tests used files around 250KB.

What it tests:

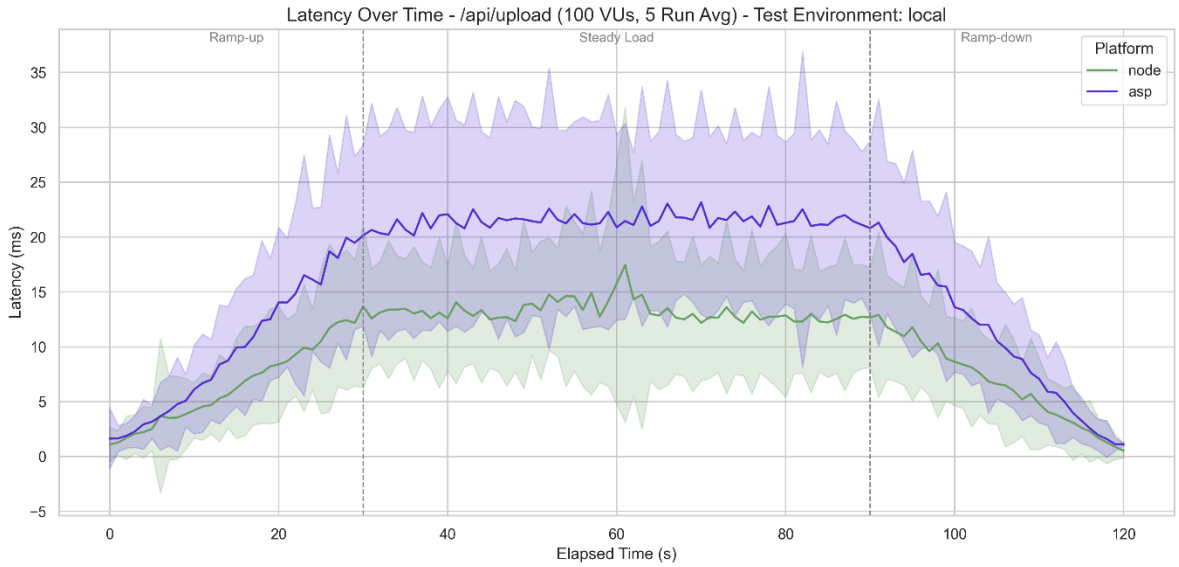
- **Request parsing throughput**
- **Disk write latency**
- **Multipart boundary handling and memory usage**

Academic Relevance:

- **Matches /upload endpoint in [Tanadechopon et al., 2023]**
- **Important for applications handling documents, images, etc.**

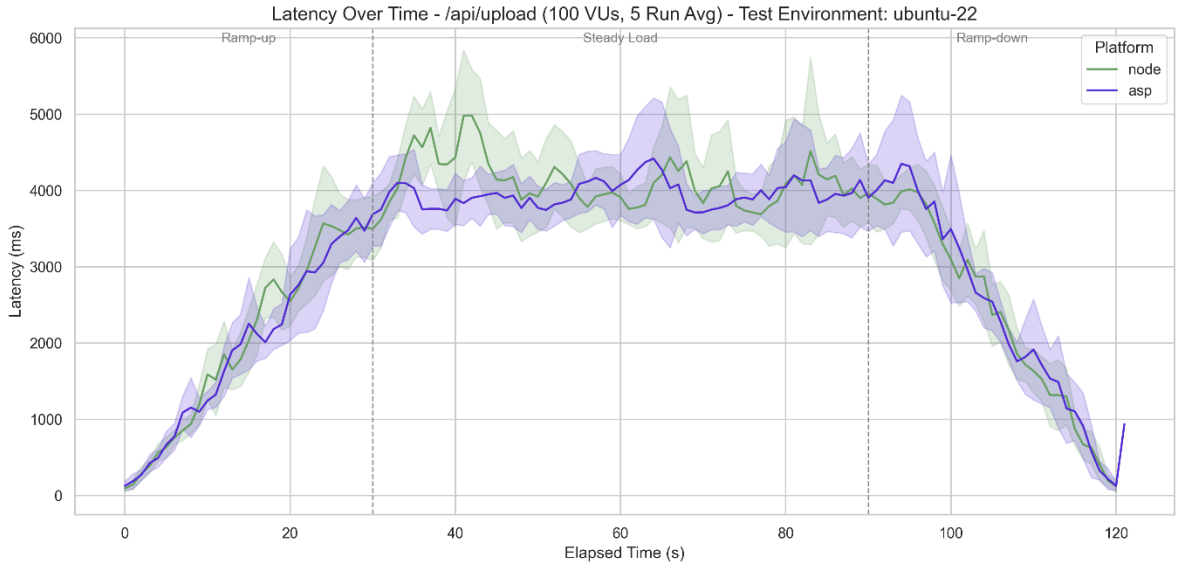
Citation Pinpoint:
Tanadechopon et al., 2023, Section III, /upload, Table III

Local:



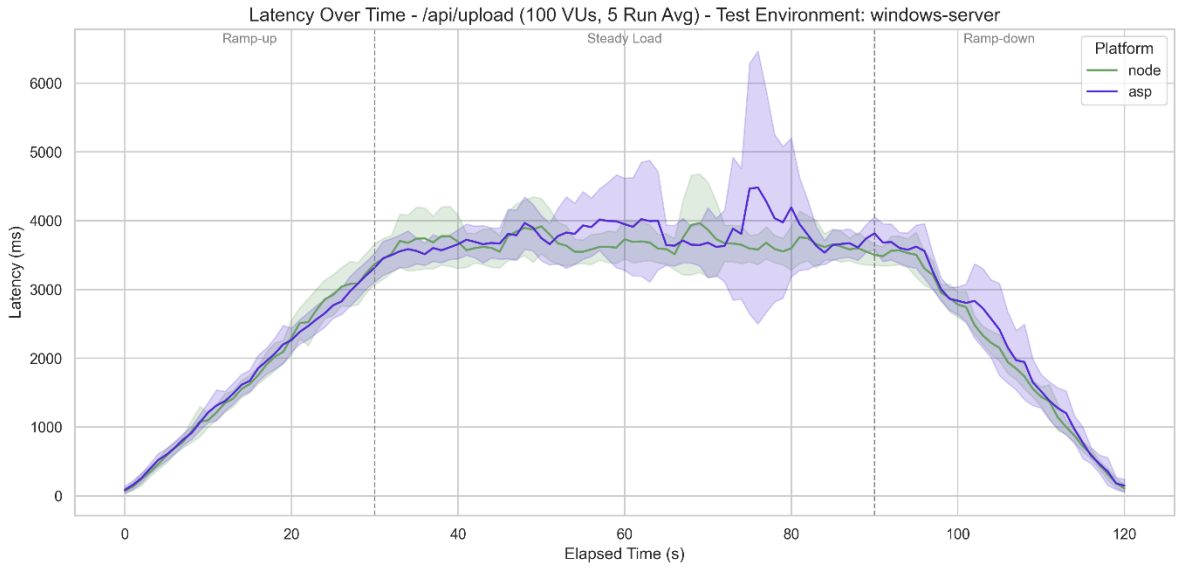
Platform	Count	Mean (ms)	Std Dev (ms)	Min (ms)	Max (ms)	p95 (ms)	p99 (ms)	Error Rate (%)
asp	278,182	15.90	10.38	0.76	122.22	35.99	48.03	0.00
node	434,085	10.21	6.79	0.27	121.32	23.22	31.36	0.00

Ubuntu:



Platform	Count	Mean (ms)	Std Dev (ms)	Min (ms)	Max (ms)	p95 (ms)	p99 (ms)	Error Rate (%)
asp	7,341	3,120.00	1,302.13	23.23	6,527.90	4,780.50	5,455.18	0.00
node	7,201	3,176.57	1,386.54	30.09	7,468.03	5,095.53	5,877.61	0.00

Windows:



Platform	Count	Mean (ms)	Std Dev (ms)	Min (ms)	Max (ms)	p95 (ms)	p99 (ms)	Error Rate (%)
asp	7,982	2,862.58	1,291.27	26.37	9,498.83	4,234.69	5,820.71	0.00
node	8,246	2,766.66	1,208.06	29.01	6,061.95	4,148.40	4,683.63	0.00

Upload Endpoint Analysis – /api/upload (POST – 250KB File)

The /api/upload endpoint accepts a 250KB binary file via a multipart POST request. This test evaluates server-side handling of incoming file streams, form parsing, and disk write operations. It is primarily **I/O-bound**, with minimal CPU involvement. The test was executed with 100 Virtual Users (VUs) over 5 runs, across three environments: a MacBook Pro (M1 Pro), and a virtual server (4 vCPU, 8GB RAM, SSD) running Ubuntu 22 and Windows Server 2022.

Key Results:

Metric	Local (MacBook Pro)	Ubuntu VDS	Windows Server VDS
ASP.NET Mean (ms)	15.90	3,120.00	2,862.58
Node.js Mean (ms)	10.21	3,176.57	2,766.66
ASP.NET p99 (ms)	48.03	5,455.18	5,820.71
Node.js p99 (ms)	31.36	5,877.61	4,683.63
ASP.NET Error Rate (%)	0.00	0.00	0.00
Node.js Error Rate (%)	0.00	0.00	0.00
ASP.NET Count	278,182	7,341	7,982
Node.js Count	434,085	7,201	8,246

Conclusion:

For small binary file uploads, both platforms exhibit **comparable performance** in production-like environments. On Ubuntu and Windows, **Node.js shows slightly lower mean latency and better max-tail latency**, while **ASP.NET demonstrates higher request stability in some runs but occasionally suffers from p99 spikes**. Importantly, neither framework failed under pressure, with all requests processed successfully.

Although Node.js performed better in the local tests, this result carries limited weight in production relevance. Overall, for medium-scale file uploads, **both ASP.NET and Node.js are viable options**, with **no major architectural disadvantage** observed on either side.

/api/login (POST)

Description:
Authenticates a user by querying users table and comparing the provided password using bcrypt.

What it tests:

- **I/O latency from DB read**
- **CPU cost of password hashing**
- **Realistic user auth flow (including error and success cases)**

Academic Relevance:

- **Modeled after /login in [Tanadechopon et al.]**
- **Combines CPU-bound and DB-read, making it a hybrid workload**

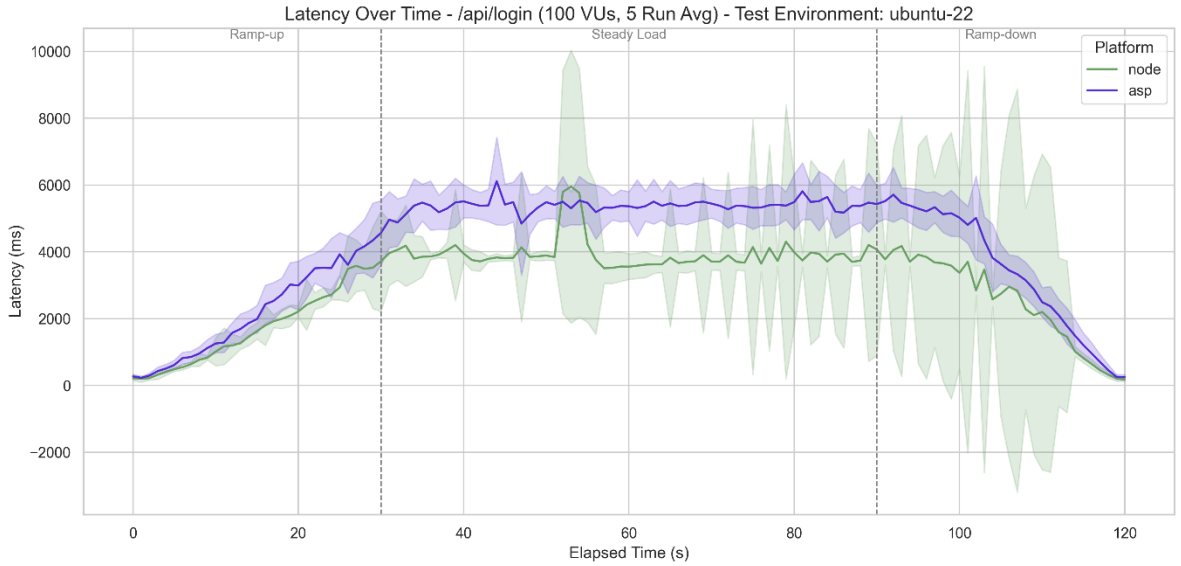
Citation Pinpoint:
Tanadechopon et al., 2023, Section III, /login, Table III

Local:



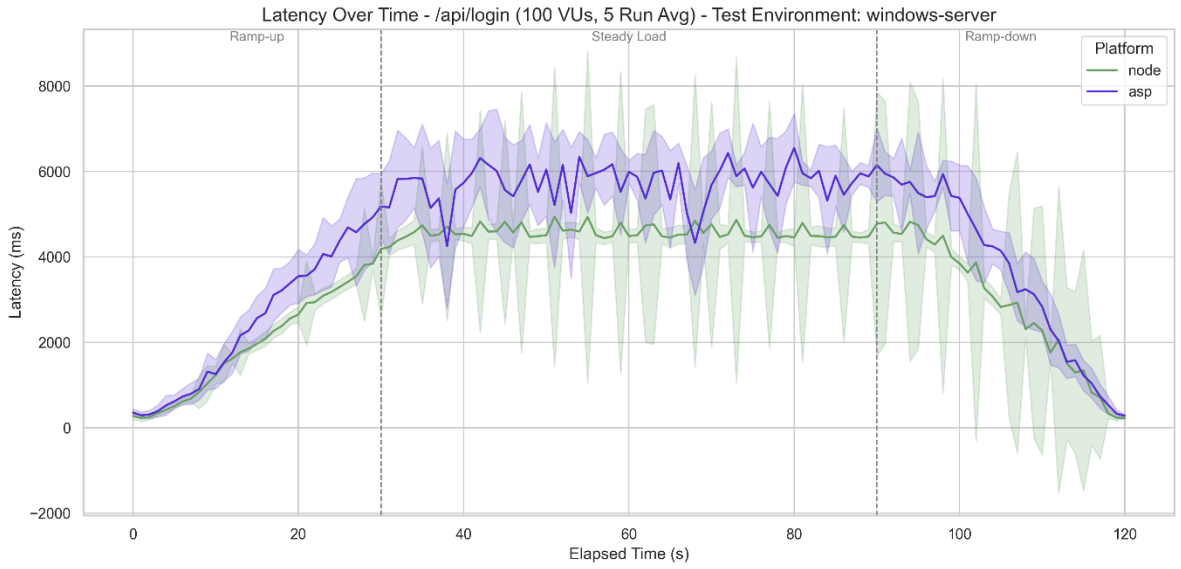
Platform	Count	Mean (ms)	Std Dev (ms)	Min (ms)	Max (ms)	p95 (ms)	p99 (ms)	Error Rate (%)
asp	69,784	647.29	267.56	70.21	1,429.34	938.96	1,054.91	0.00
node	34,189	1,324.77	552.67	66.99	13,359.68	1,757.36	1,762.10	0.00

Ubuntu:



Platform	Count	Mean (ms)	Std Dev (ms)	Min (ms)	Max (ms)	p95 (ms)	p99 (ms)	Error Rate (%)
asp	11,373	4,031.77	1,907.70	142.34	11,191.61	6,204.10	7,334.34	0.00
node	15,299	2,988.06	2,446.18	111.40	52,748.47	4,042.15	9,493.89	0.00

Windows:



Platform	Count	Mean (ms)	Std Dev (ms)	Min (ms)	Max (ms)	p95 (ms)	p99 (ms)	Error Rate (%)
asp	10,603	4,329.29	2,110.98	156.94	10,087.67	6,892.17	7,493.31	0.00
node	13,302	3,441.83	2,146.36	128.24	46,971.69	4,760.21	4,919.86	0.00

Login Endpoint Analysis – /api/login (POST)

The /api/login endpoint simulates an authentication request with logic such as credential validation, password hashing comparison (e.g., bcrypt), and token/session generation. These operations can be moderately CPU-bound and involve short bursts of memory and I/O. The test was executed with 100 Virtual Users (VUs), averaged over 5 runs, on a local M1 Pro MacBook and a VDS running Ubuntu 22 and Windows Server 2022.

Key Results:

Metric	Local (MacBook Pro)	Ubuntu VDS	Windows Server VDS
ASP.NET Mean (ms)	647.29	4,031.77	4,329.29
Node.js Mean (ms)	1,324.77	2,988.06	3,441.83
ASP.NET p99 (ms)	1,054.91	7,334.34	7,493.31
Node.js p99 (ms)	1,762.10	9,493.89	4,919.86
ASP.NET Error Rate (%)	0.00	0.00	0.00
Node.js Error Rate (%)	0.00	0.00	0.00
ASP.NET Count	69,784	11,373	10,603
Node.js Count	34,189	15,299	13,302

Conclusion:

The login test reveals interesting behavior across platforms. While **ASP.NET handled authentication more efficiently on macOS**, **Node.js pulled ahead slightly on Ubuntu**, likely due to lower overhead in native bcrypt implementations or differences in internal thread pool handling. On Windows Server, results were closer, though ASP.NET showed greater latency stability, and Node.js exhibited better p99 performance.

Since login endpoints often include password hashing (e.g., bcrypt with cost factors of 10–12), these results reflect the **impact of synchronous CPU-bound hashing functions** under concurrency. While both platforms successfully processed all requests, **Node.js may require offloading strategies (e.g., worker threads) for scalable authentication**, whereas **ASP.NET manages concurrency more natively** with minimal configuration.

/api/customers (GET)

Description:
Fetches the first 10 rows from the customers table in the database.

What it tests:

- **Lightweight read query performance**
- **Overhead of DB connection, query parsing, and JSON serialization**

Academic Relevance:

- **Similar to /data?n=100 in [Tanadechopon et al., 2023]**
- **Useful for baseline DB read performance**

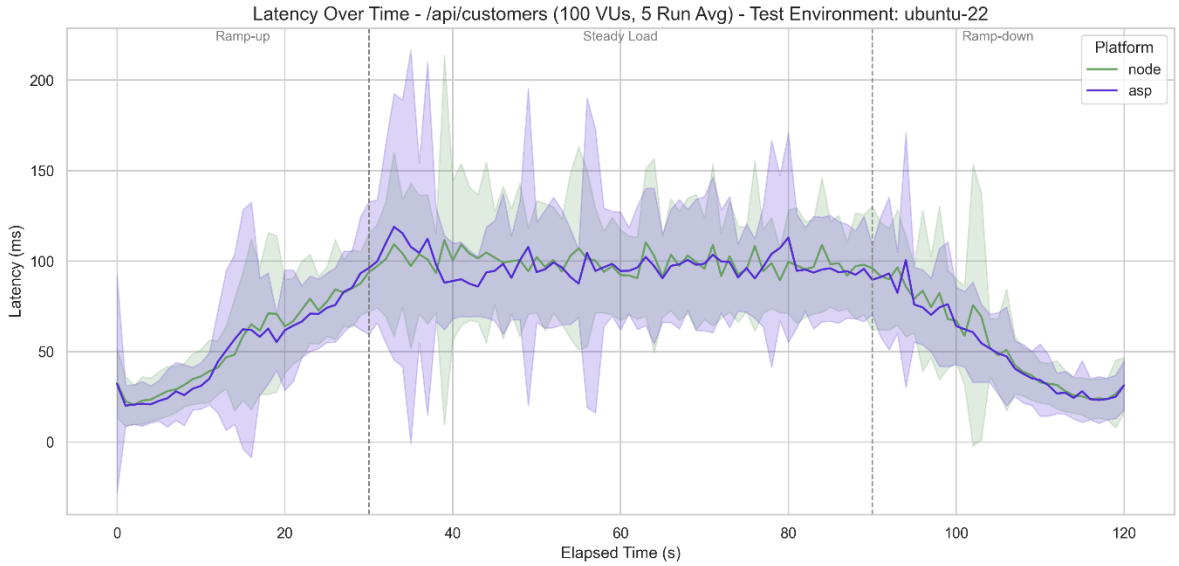
Citation Pinpoint:
Tanadechopon et al., 2023, Section III, /data, Table III & IV.

Local:



Platform	Count	Mean (ms)	Std Dev (ms)	Min (ms)	Max (ms)	p95 (ms)	p99 (ms)	Error Rate (%)
asp	3,438,573	2.59	1.84	0.12	118.92	4.06	9.87	0.00
node	2,561,311	3.49	1.71	0.08	60.87	4.87	8.44	0.00

Ubuntu:



Platform	Count	Mean (ms)	Std Dev (ms)	Min (ms)	Max (ms)	p95 (ms)	p99 (ms)	Error Rate (%)
asp	577,591	77.91	44.42	7.07	1,002.88	141.77	220.65	0.00
node	559,151	80.48	41.12	7.17	848.17	144.55	205.08	0.00

Windows:



Platform	Count	Mean (ms)	Std Dev (ms)	Min (ms)	Max (ms)	p95 (ms)	p99 (ms)	Error Rate (%)
asp	1,610,439	27.91	21.72	4.91	940.54	46.82	97.00	0.00
node	726,865	61.91	30.43	4.96	768.95	101.46	144.34	0.00

Customers Endpoint Analysis – /api/customers (GET – 10 Rows)

The /api/customers endpoint represents a **lightweight read operation** that fetches a small number of records (10 rows) from a single database table. It is primarily I/O-bound, with minimal CPU or memory overhead. This makes it a strong indicator of **baseline latency**, **framework overhead**, and **request-response consistency** under concurrent load. The tests were conducted using 100 Virtual Users (VUs) over 5 runs across three environments: a local M1 Pro MacBook, and a virtual server (4-core CPU, 8GB RAM, SSD) running Ubuntu 22 and Windows Server 2022.

Key Results:

Metric	Local (MacBook Pro)	Ubuntu VDS	Windows Server VDS
ASP.NET Mean (ms)	2.59	77.91	27.91
Node.js Mean (ms)	3.49	80.48	61.91
ASP.NET p99 (ms)	9.87	220.65	97.00
Node.js p99 (ms)	8.44	205.08	144.34
ASP.NET Error Rate (%)	0.00	0.00	0.00
Node.js Error Rate (%)	0.00	0.00	0.00
ASP.NET Count	3,438,573	577,591	1,610,439
Node.js Count	2,561,311	559,151	726,865

Conclusion:

The /api/customers test highlights each framework's raw efficiency in handling **frequent, low-cost read operations**. ASP.NET slightly outperforms Node.js in every environment, but the most significant gap is seen on **Windows Server**, where it handled over **twice the number of requests** and maintained lower latency throughout. On Ubuntu, both platforms performed nearly identically, reflecting strong I/O capabilities and optimized DB drivers.

These findings reaffirm that **for high-frequency, low-cost GET endpoints**, both platforms are viable, but **ASP.NET delivers more throughput and tighter latency bounds**, particularly in environments aligned with the .NET ecosystem.